

Taller 3 - Programación discreta: documento de explicación

Matemáticas Discretas I · Universidad Nacional de Colombia Docente: Jhoan Sebastian Tenjo García

Este documento explica, para cada uno de los 10 ejercicios, qué problema resuelve el programa, qué idea matemática usa, cómo se ejecuta, qué pruebas se hicieron y qué limitaciones tiene la solución. El código completo, comentado, está en la carpeta `src/` y las pruebas en `tests/`.

Bloque A. Criptografía

Ejercicio 1 — Cifrado César (`src/cripto/cesar.py`)

Problema que resuelve. Cifrar y descifrar un texto desplazando cada letra una cantidad fija k , y romper un cifrado cuando k es desconocido, probando las 26 posibilidades.

Idea matemática. El alfabeto A..Z se identifica con $\mathbb{Z}_{26} = \{0, \dots, 25\}$. Cifrar es la función $f(x) = (x + k) \bmod 26$. Como f es una biyección de $\mathbb{Z}_{26}$ en sí mismo, tiene inversa $f^{-1}(y) = (y - k) \bmod 26$: por eso descifrar usa el desplazamiento contrario, $-k$. El ataque de fuerza bruta es posible porque el espacio de llaves es diminuto (solo 26 valores): probarlos todos es computacionalmente trivial, a diferencia de un cifrado con una llave de, por ejemplo, 128 bits.

Cómo se ejecuta.

```
python -m src.cripto.cesar
```

Funciones principales: `cifrar(texto, k)`, `descifrar(texto, k)`, `fuerza_bruta(texto)`.

Qué pruebas se hicieron (`tests/test_cesar.py`, 5 pruebas). Se verificó el ejemplo del enunciado (HOLA UNAL con $k=3 \rightarrow$ KROD XQDO), que cifrar y descifrar son inversos para varios k , que espacios/números/puntuación quedan intactos, que mayúsculas y minúsculas se respetan por separado, y que la fuerza bruta efectivamente encuentra el k correcto entre las 26 opciones.

Limitaciones. Solo cubre el alfabeto latino sin "ñ" (tal como pide el enunciado). No es un cifrado seguro: se incluye únicamente con fines didácticos.

Ejercicio 2 — RSA de juguete (`src/cripto/rsa_toy.py`)

Problema que resuelve. A partir de dos primos p , q y un exponente público e , generar las llaves RSA y cifrar/descifrar un mensaje M .

Idea matemática. $n = p \cdot q$, y $\phi(n) = (p-1)(q-1)$ cuenta los enteros en $[1, n-1]$ coprimos con n . El exponente privado d es el inverso de e módulo $\phi(n)$, calculado con el **algoritmo de Euclides extendido**, que además de $\gcd(a, b)$ entrega coeficientes x , y tales que $a \cdot x + b \cdot y = \gcd(a, b)$ (identidad de Bézout). El cifrado es $C \equiv M^e \pmod{n}$ y el descifrado $M \equiv C^d \pmod{n}$. Esto funciona por el teorema de Euler: si $\gcd(e, \phi(n)) = 1$, existe d con $e \cdot d \equiv 1 \pmod{\phi(n)}$, es decir $e \cdot d = 1 + t \cdot \phi(n)$ para algún entero t , y entonces $C^d = M^{(e \cdot d)} = M \cdot (M^{\phi(n)})^t \equiv M \pmod{n}$ cuando $\gcd(M, n) = 1$, pues $M^{\phi(n)} \equiv 1 \pmod{n}$.

Cómo se ejecuta.

```
python -m src.cripto.rsa_toy
```

`generar_llaves(p, q, e)` valida que p , q sean primos distintos y que $\gcd(e, \phi(n)) = 1$ (si no, lanza `ValueError`); `cifrar(m, e, n)` y `descifrar(c, d, n)` usan exponenciación modular rápida (`pow(base, exp, mod)`), parte del lenguaje, no una librería que resuelva RSA).

Qué pruebas se hicieron (`tests/test_rsa_toy.py`, 6 pruebas). Se reprodujo exactamente el caso obligatorio del enunciado ($p=61$, $q=53$, $e=17 \Rightarrow n=3233$, $\phi=3120$, $d=2753$, $C=2790$, y M se recupera igual a 65); se probó el ciclo cifrado→descifrado con otro par de primos y varios mensajes; se probó que un e inválido ($\gcd(e, \phi(n)) \neq 1$) lanza error; y se verificaron por separado el algoritmo de Euclides extendido (identidad de Bézout) y el inverso modular.

Limitaciones. Usa primos pequeños de juguete: **no es seguro para uso real** (RSA real usa primos de cientos de dígitos). No implementa relleno (padding) criptográfico.

Ejercicio 3 — MPC básico: suma secreta (`src/cripto/mpc_promedio.py`)

Problema que resuelve. Calcular la suma y el promedio de una lista de notas sin que ningún servidor individual conozca las notas originales.

Idea matemática. Es un esquema de reparto de secretos aditivo (*additive secret sharing*) módulo M . Cada nota x se reparte en tres partes aleatorias s_1 , s_2 en $\mathbb{Z}_M$ (elegidas uniformemente al azar) y $s_3 = (x - s_1 - s_2) \bmod M$, de modo que $x \equiv s_1 + s_2 + s_3 \pmod{M}$. Como s_1 y s_2 son uniformemente aleatorias, cualquier valor de x es igual de consistente con una parte vista de forma aislada: una sola parte no filtra información sobre x (es un "one-time pad" aplicado a la suma). Sumar las partes componente a componente conmuta con la suma de las notas, así que cada servidor puede agregar sus propias partes en un total parcial, y sumar los tres totales parciales reconstruye la suma real sin que nadie haya visto una nota individual.

Cómo se ejecuta.

```
python -m src.cripto.mpc_promedio
```

`repartir_notas(notas, M)` reparte toda la lista; `suma_parcial_servidor` calcula el total que ve cada servidor; `reconstruir_suma` combina los tres totales; `simular_protocolo` encadena todo el proceso.

Qué pruebas se hicieron (`tests/test_mpc_promedio.py`, 6 pruebas). Se verificó el ejemplo del enunciado (`[40, 35, 50, 25] => suma 150, promedio 37.5`); que el reparto es correcto para distintas notas repitiendo el experimento muchas veces (para cubrir la aleatoriedad); que funciona con listas de tamaño arbitrario; que una parte vista aislada toma muchos valores distintos entre corridas (evidencia de que no hay un patrón fijo); y que se valida el rango de la nota (0 a 50).

Limitaciones. Es una simulación educativa de la idea de MPC, no un protocolo criptográfico auditado; asume que los "servidores" no se coluden (si los tres servidores comparten sus partes entre sí, sí podrían reconstruir la nota individual).

Bloque B. Grafos

Ejercicio 4 — Ruta más corta con Dijkstra (`src/grafos/dijkstra.py`, `src/grafos/grafos.py`)

Problema que resuelve. Encontrar la distancia y la ruta más corta entre dos puntos de una red de transporte representada como grafo ponderado.

Idea matemática. Dijkstra mantiene `dist[v]`, la mejor distancia conocida desde el origen a cada vértice, y una cola de prioridad. En cada paso extrae el vértice no visitado con menor `dist[v]` y relaja sus aristas: si `dist[u] + peso(u, v) < dist[v]`, actualiza `dist[v]`. La demostración de que el vértice extraído ya tiene su distancia final depende de que **todos los pesos sean no negativos**: si existieran pesos negativos, un camino con más aristas podría terminar siendo más corto que uno "greedy" ya cerrado, y el algoritmo dejaría de ser correcto. Un camino es óptimo cuando ninguna otra secuencia de aristas entre los mismos dos vértices tiene menor suma de pesos.

Cómo se ejecuta.

```
python -m src.grafos.dijkstra
```

El grafo de prueba (`grafo_ciudad_prueba()` en `grafos.py`) tiene 9 vértices y 13 aristas (cumple el mínimo de 8 y 12 pedido). También se puede cargar un grafo desde JSON con `Grafo.desde_archivo("ruta.json")`.

Qué pruebas se hicieron (`tests/test_dijkstra.py`, 6 pruebas). Se verificó que el grafo de prueba cumple el tamaño mínimo; que la distancia de un vértice a sí mismo es 0; que la ruta reportada es consistente (la suma de los pesos de sus aristas es igual a la distancia devuelta); un caso pequeño resuelto a mano donde el camino indirecto es mejor que el directo; que un vértice inalcanzable da distancia infinita y ruta vacía; y que un peso negativo es rechazado explícitamente.

Limitaciones. Solo funciona con pesos no negativos (por diseño, según el enunciado). La implementación usa `heapq`, adecuada para grafos pequeños/medianos como los del taller.

Ejercicio 5 — Cierre de una estación (`src/grafos/cierre_estacion.py`)

Problema que resuelve. Medir cómo cambian las rutas más cortas de varios pares origen-destino cuando se cierra un vértice (o una arista) de la red.

Idea matemática. Cerrar un vértice v equivale a trabajar con el subgrafo inducido $G' = G - v$ (se eliminan v y todas sus aristas incidentes). Para cada par se compara `dist_G(origen, destino)` con `dist_G'(origen, destino)`: si el destino queda inalcanzable, el estado es "desconectado"; si la distancia aumenta, el cierre obligó a un camino más largo; si no cambia, existía una ruta alterna igual de corta que no pasaba por v .

Cómo se ejecuta.

```
python -m src.grafos.cierre_estacion
```

La demo cierra el vértice "Centro" del grafo de prueba y evalúa 5 pares origen-destino, imprimiendo una tabla con columnas origen, destino, distancia antes, distancia después, diferencia y estado.

Qué pruebas se hicieron (`tests/test_cierre_estacion.py`, 5 pruebas). Se probó el cierre de un vértice con 5 pares (la distancia nunca puede *disminuir* tras un cierre); un caso construido a mano donde el cierre desconecta por completo dos vértices (único puente); un caso donde existe una ruta alterna igual de corta y el estado debe ser "sin cambio"; el cierre de una arista en vez de un vértice; y que pedir cero o dos cierres a la vez lanza error.

Limitaciones. Solo se puede cerrar un elemento (un vértice o una arista) a la vez por llamada; para simular cierres múltiples habría que encadenar llamadas.

Ejercicio 6 — Coloreo de grafos (`src/grafos/coloreo.py`)

Problema que resuelve. Asignar franjas horarias de examen (colores) a cursos (vértices) de forma que dos cursos con estudiantes en común (arista) nunca coincidan.

Idea matemática. Una coloración propia de $G=(V, E)$ asigna a cada vértice un color tal que $c(u) \neq c(v)$ para toda arista $\{u, v\}$. El algoritmo **voraz** recorre los vértices en algún orden y da a cada uno el menor color no usado por sus vecinos ya coloreados. Esto siempre produce una coloración válida por construcción, pero **no garantiza** usar el número cromático mínimo: el resultado depende del orden de recorrido, y existen órdenes que fuerzan usar más colores de los estrictamente necesarios (por ejemplo, un grafo bipartito recorrido en el peor orden puede terminar usando varios colores en vez de solo 2).

Cómo se ejecuta.

```
python -m src.grafos.coloreo
```

`grafo_cursos_prueba()` define 10 cursos con 14 conflictos; `coloreo_voraz` asigna los colores y `es_coloreo_valido` confirma que ningún par adyacente comparte color.

Qué pruebas se hicieron (`tests/test_coloreo.py`, 6 pruebas). Se verificó el tamaño mínimo del grafo de prueba; que su coloreo voraz es válido y cubre todos los vértices; que un triángulo (3 vértices todos conectados entre sí) necesita exactamente 3 colores; que un grafo sin ninguna arista usa un solo color; que la función de verificación detecta correctamente una coloración inválida armada a mano; y el agrupamiento de vértices por color.

Limitaciones. El algoritmo es voraz, no óptimo: no calcula el número cromático exacto, solo una cota superior válida (que es justamente lo que pide el enunciado).

Bloque C. Álgebra de Boole, Shannon y computación cuántica

Ejercicio 7 — Tablas de verdad y circuitos lógicos (`src/boole/tablas_verdad.py`)

Problema que resuelve. Generar la tabla de verdad completa de una expresión booleana con variables

A, B, C, D y los conectivos AND, OR, NOT, XOR, y evaluarla en una entrada concreta.

Idea matemática. Una expresión con n variables define una función $f: \{0,1\}^n \rightarrow \{0,1\}$. La tabla de verdad enumera exhaustivamente las 2^n combinaciones de entrada y el valor de f en cada una. Cada fila corresponde exactamente a una configuración de un circuito digital: AND es una compuerta "Y", OR una compuerta "O", NOT un inversor y XOR una compuerta de "o exclusivo" — por eso una tabla de verdad ES la especificación funcional de un circuito lógico. El evaluador es un parser recursivo propio (no usa eval de Python) con la precedencia usual NOT > AND > XOR > OR.

Cómo se ejecuta.

```
python -m src.boole.tablas_verdad
```

Las tres expresiones obligatorias del enunciado están en EXPRESIONES_EJEMPLO: (A AND B) OR (NOT C), (A XOR B) AND C, (A OR B) AND (NOT A OR C).

Qué pruebas se hicieron (tests/test_tablas_verdad.py, 6 pruebas). Se comprobaron las 8 combinaciones completas de la primera expresión contra el resultado calculado con los operadores nativos de Python; varios casos puntuales de las otras dos expresiones; que la tabla tiene 2^n filas; que una expresión con un carácter no válido lanza error; y una prueba de regresión específica: el extractor de variables no debe confundir la "A" y la "D" que aparecen *dentro de la palabra clave* "AND" con variables reales (bug real que se encontró y corrigió durante el desarrollo — ver commits).

Limitaciones. El evaluador solo reconoce variables A-D, los conectivos en mayúsculas AND/OR/NOT/XOR y paréntesis; no soporta operadores adicionales (como implicación) ni más de 4 variables (aunque el diseño se podría extender fácilmente).

Ejercicio 8 — Simplificación booleana con Quine-McCluskey (src/boole/simplificacion.py)

Problema que resuelve. A partir de los minterminos de una función booleana de 3 o 4 variables, producir una expresión simplificada en forma suma de productos (menos compuertas que la forma canónica).

Idea matemática. Un mintermino es una fila de la tabla de verdad donde la función vale 1, escrita como el producto de las n variables (cada una directa o negada según su bit). Dos términos que difieren en un único bit se pueden combinar por la ley $(X \cdot V) + (X \cdot \neg V) = X$, eliminando la variable que cambia. Repitiendo la combinación hasta que ningún par más se pueda fusionar se obtienen los **implicantes primos**. Luego se seleccionan implicantes primos —dando prioridad a los **esenciales** (los únicos que cubren cierto mintermino)— hasta cubrir todos los minterminos originales; la OR de esos implicantes es la expresión simplificada. Dos expresiones son equivalentes si y solo si tienen la misma tabla de verdad, así que el programa reconstruye ambas tablas y las compara para confirmar la simplificación.

Cómo se ejecuta.

```
python -m src.boole.simplificacion
```

simplificar(minterminos, n_vars) devuelve la expresión; verificar_equivalencia hace la comprobación de tabla de verdad.

Qué pruebas se hicieron (tests/test_simplificacion.py, 5 pruebas). El caso sugerido del enunciado, minterminos {1,3,5,7} con 3 variables, simplifica correctamente a una expresión equivalente a C (todos esos minterminos tienen el bit menos significativo en 1); una función de 4 variables que no depende de una de ellas; los casos extremos de función constante falsa (sin minterminos) y constante verdadera (todos los minterminos); y un único mintermino. En todos los casos se verifica la igualdad de tabla de verdad entre el original y la simplificación.

Limitaciones. La selección de implicantes primos usa una heurística (esenciales primero, luego voraz por cobertura): para funciones con múltiples soluciones

óptimas empatadas, no siempre entrega la de menor número de términos en casos ambiguos poco comunes, aunque el resultado es siempre una expresión **correcta** y equivalente.

Ejercicio 9 — Entropía de Shannon (src/informacion/shannon.py)

Problema que resuelve. Medir cuánta información (incertidumbre) contiene un texto, y comparar dos textos distintos.

Idea matemática. Si un símbolo i aparece con probabilidad p_i (frecuencia relativa en el texto), la entropía de Shannon $H = - \sum p_i \log_2(p_i)$ mide, en bits, la incertidumbre promedio de adivinar el siguiente símbolo. Si un símbolo aparece siempre ($p_i=1$), no hay sorpresa y $H=0$; si todos los símbolos son igual de probables, no hay forma de adivinar mejor que al azar y H alcanza su máximo, $\log_2(\text{número de símbolos})$. H mide la **forma** de la distribución de frecuencias, no la longitud del texto: por eso "AAAAAAAAA" y "AAAAA" tienen exactamente la misma entropía (0 bits).

Cómo se ejecuta.

```
python -m src.informacion.shannon
```

`entropia(texto)` calcula H ; `comparar(texto_a, texto_b)` explica en una frase cuál mensaje es más incierto y por qué.

Qué pruebas se hicieron (tests/test_shannon.py, 7 pruebas). Un texto de un solo símbolo repetido da entropía exactamente 0; dos símbolos equiprobables dan entropía 1 bit; cuatro símbolos equiprobables dan 2 bits (casos con solución matemática exacta conocida); un texto repetitivo comparado contra uno variado (con más símbolos distintos) confirma que el segundo tiene mayor entropía; las probabilidades calculadas suman 1; las frecuencias se cuentan correctamente; y un texto vacío lanza error.

Limitaciones. Trata cada carácter como un símbolo independiente (entropía de orden cero): no captura correlaciones entre caracteres consecutivos (para eso se necesitaría entropía condicional o un modelo de Markov). La extensión de Huffman mencionada en el enunciado es opcional y no se implementó.

Ejercicio 10 — Simulador cuántico de un qubit (src/cuantica/qubit_simulador.py)

Problema que resuelve. Simular el estado de un solo qubit, aplicar las compuertas X, Z, H, calcular probabilidades de medición y simular 1000 mediciones.

Idea matemática. Un qubit se representa como un vector columna de amplitudes complejas $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ con $|\alpha|^2 + |\beta|^2 = 1$. Las compuertas de un qubit son matrices unitarias 2x2 que actúan por multiplicación matriz-vector: $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ (intercambia $|0\rangle$ y $|1\rangle$), $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ (invierte el signo de la amplitud de $|1\rangle$) y $H = (1/\sqrt{2}) \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ (crea superposición). La **regla de Born** dice que $P(\text{medir } 0) = |\alpha|^2$ y $P(\text{medir } 1) = |\beta|^2$; el simulador de mediciones muestrea de esa distribución con `random.random()`, sin usar ningún hardware ni librería cuántica.

Cómo se ejecuta.

```
python -m src.cuantica.qubit_simulador
```

`aplicar_compuerta(matriz, estado)` hace la multiplicación; `probabilidades(estado)` aplica la regla de Born; `simular_mediciones(estado, n)` repite la medición n veces.

Qué pruebas se hicieron (tests/test_qubit_simulador.py, 7 pruebas). Los tres casos obligatorios del enunciado: $X|0\rangle = |1\rangle$; $H|0\rangle$ da probabilidades exactamente 50%/50%; y $HH|0\rangle = |0\rangle$ (salvo error numérico de punto flotante). Además, las probabilidades de los estados puros $|0\rangle$ y $|1\rangle$; el efecto de Z ; que medir 1000 veces un qubit en estado puro $|1\rangle$ da siempre 1; y que 1000 mediciones de $H|0\rangle$ caen en un margen razonable alrededor de 500/500 (con semilla fija para que la

prueba sea reproducible).

Limitaciones. Es una simulación **clásica** de la probabilidad cuántica de un solo qubit: no hay entrelazamiento (se necesitarían ≥ 2 qubits), no hay ruido de hardware ni decoherencia, y las "mediciones" son simplemente muestreo aleatorio clásico ponderado por $|\alpha|^2$ y $|\beta|^2$ – muy distinto de ejecutar el circuito en un computador cuántico real, donde la medición colapsa un estado físico y está sujeta a errores de calibración del dispositivo.